Two Number Sum

AlgoExpert · Easy · Arrays

The problem

- Given distinct integers and a target, return any pair summing to the target (empty array if none).

Intuition

- For each number x, we need to know if its partner, target minus x, has already appeared.
- A hash set answers that instantly.

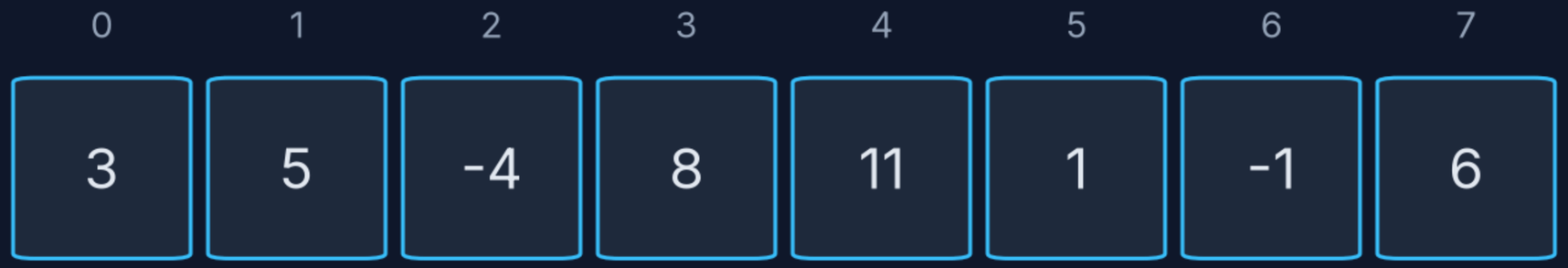
Brute force vs optimal

- Brute force: check every pair → $O(n^2)$
- Sorting + two pointers → O(n log n), O(1) space
- Hash set of seen values → O(n) time, O(n) space

Approach

- 1. Walk the array once
- 2. Check whether target minus x is in the set
- 3. If so, return the pair
- 4. Otherwise add x to the set

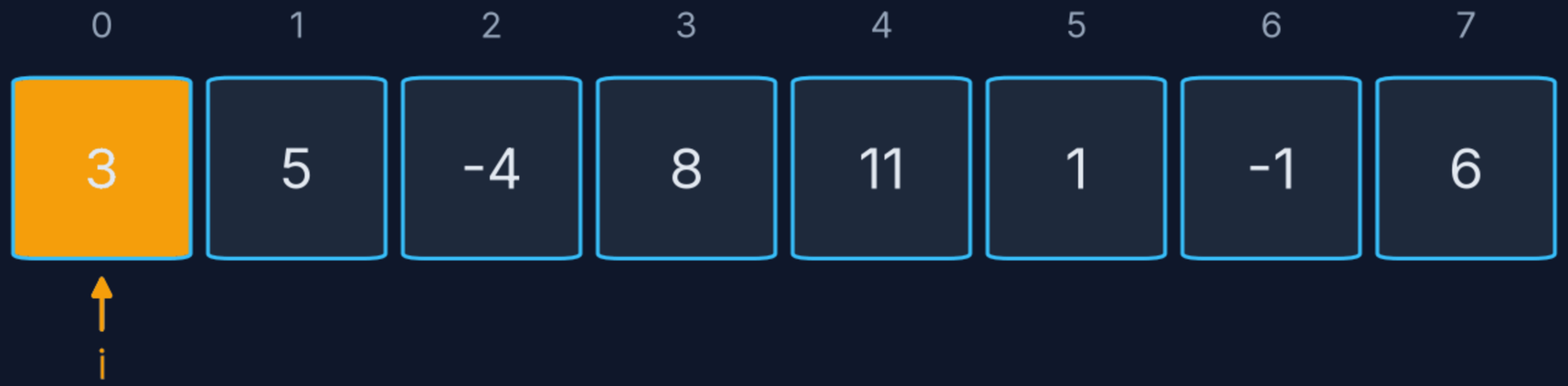
Worked example




```
target = 10
```

array = [3, 5, -4, 8, 11, 1, -1, 6], target = 10

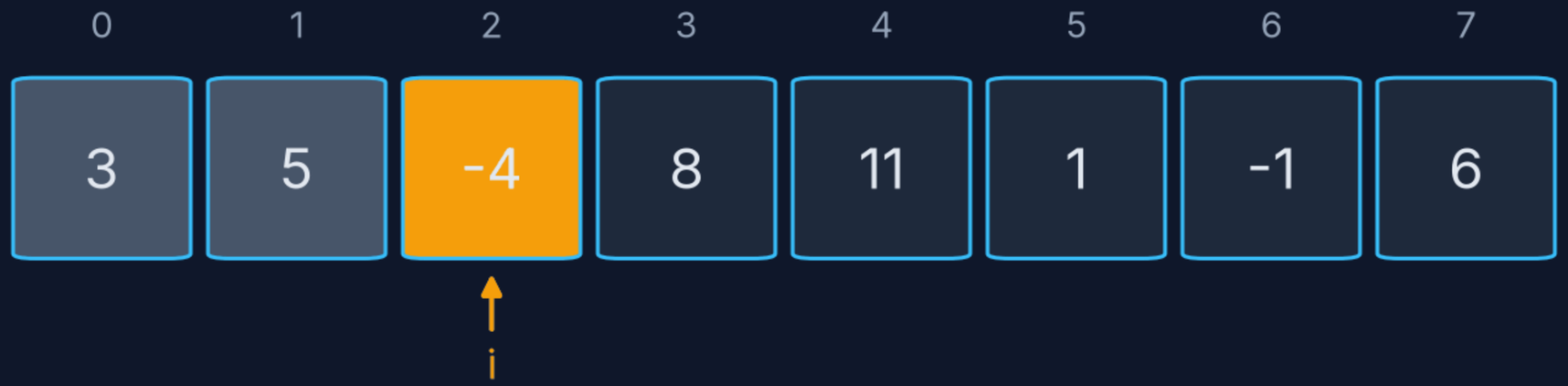
Worked example




```
x = 3, need 7
seen = {}   ✗ → add 3
```

7 not seen yet → remember 3

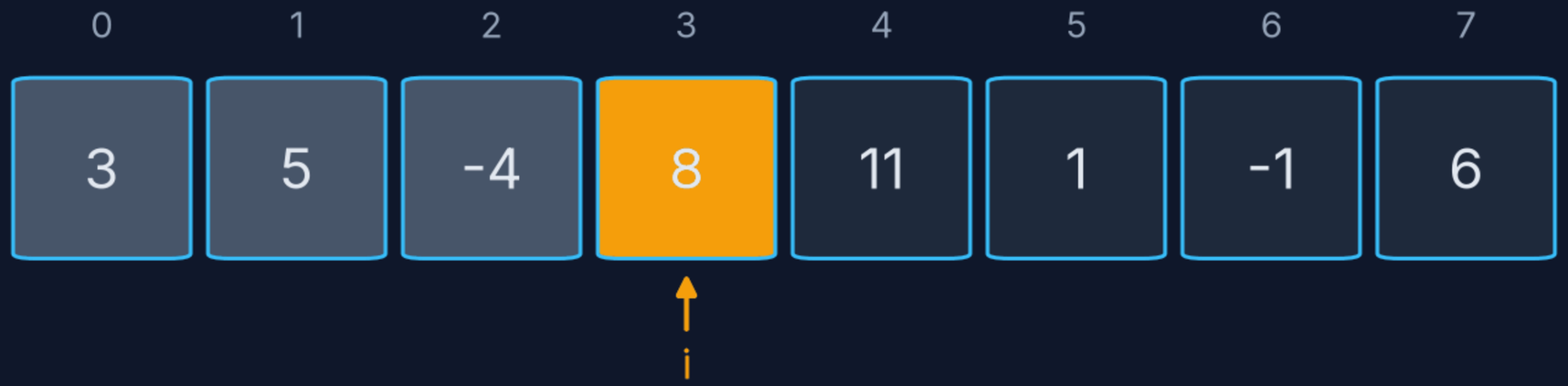
Worked example




```
x = -4, need 14
seen = {3, 5}   ✗ → add -4
```

14 not seen yet → remember -4

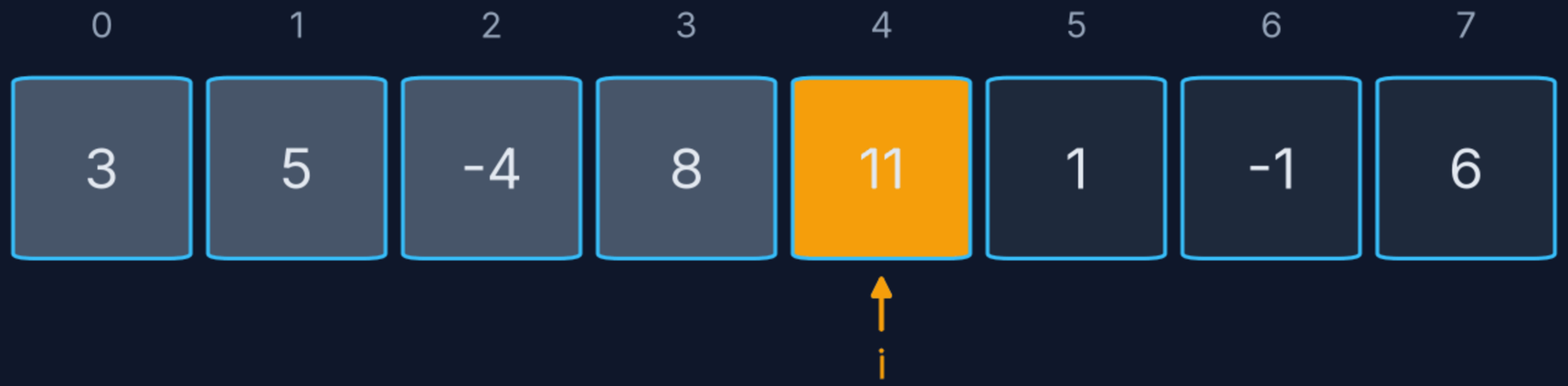
Worked example




```
x = 8, need 2
seen = {3, 5, -4}   ✗ → add 8
```

2 not seen yet → remember 8

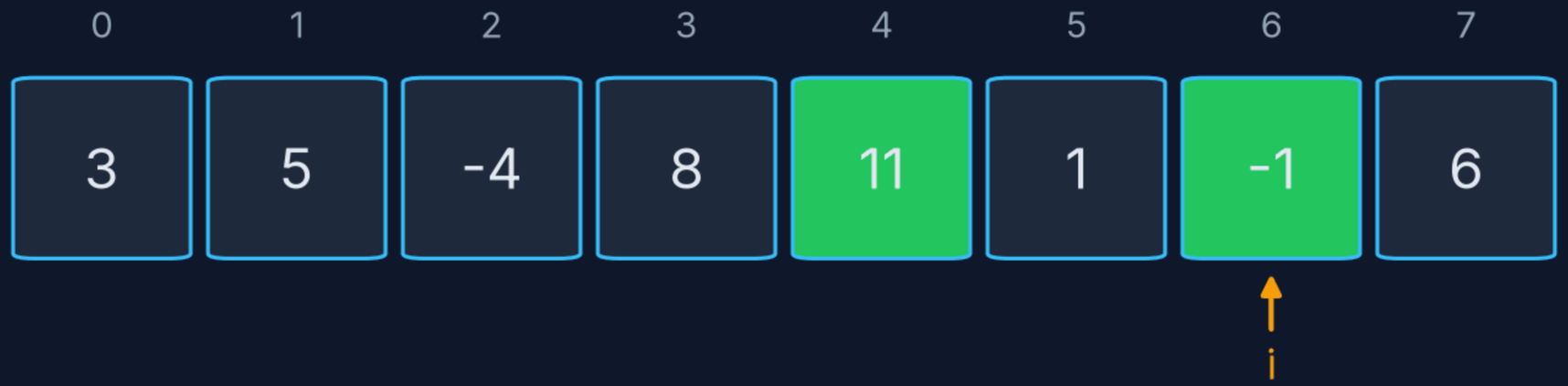
Worked example




```
x = 11, need -1
seen = {3, 5, -4, 8}   ✗ → add 11
```

-1 not seen yet → remember 11

Worked example




```
x = -1, need 10 - -1 = 11
seen = {3, 5, -4, 8, 11, 1}  ✓ contains 11
```

Found: 11 + -1 = 10

Java solution

```java
public class TwoNumberSum {
    public static int[] twoNumberSum(int[] array, int target) {
        Set<Integer> seen = new HashSet<>();
        for (int x : array) {
            if (seen.contains(target - x)) return new int[]{target - x, x};
            seen.add(x);
        }
        return new int[0];
    }
}
```

Complexity

- Time: O(n)
- Space: O(n) using a hash set of seen values

Edge cases & pitfalls

- No pair → return an empty array
- Don't pair a number with itself (check before adding)
- Negative numbers work the same way

Interview tips & follow-ups

- Mention all three approaches and their trade-offs
- Follow-up: return indices; or all pairs; or 3-sum
- Ask whether the input fits in memory

Recap

- For each number x, we need to know if its partner, target minus x, has already appeared.
- Complexity: O(n)